Spark ML - recommendation system:

40% of app installs on Google Play → come from recommendations.

60% of watch time on YouTube → comes from recommendations.

Spark ML - recommendation system:

Tasks:

https://developers.google.com/machine-learning/recommendation

1. Based on the documentation, read the materials describing - What is a recommendation system and what are the methods of their creation
2. Download a sample data set from online resources, e.g. databases of films and their ratings: https://grouplens.org/datasets/movielens/
3. Read the description of the downloaded data set.
4. Create a Spark session and read the data set downloaded in *.csv files
5. Perform the initialization of the estimator and prepare a ready-made untrained model
6. Train a model on the training set for making recommendations
7. Evaluate the model and predictions for recommendation purposes:
8. Using the developed model of the recommendation system, answer the questions:
   - which films are worth showing to specific users?
   - Specify users who will like given films:

Description of the model construction:

https://developers.google.com/machine-learning/recommendation

Question:

What is a recommendation system?

Question:

What is a recommendation system?

Description of the model construction:

https://developers.google.com/machine-learning/recommendation

The rapid increase in data collection has led to a situation in which users (or systems) are unable to respond to the changing offer on an ongoing basis.

..question : → … Why?

Answer : → because the multitude of information leads to information overload and results in many choice alternatives that are not always well suited to the user's preferences.

Recommendation systems are a type of information filtering systems, they improve the quality of search results and provide results that are more relevant to the search criteria or related to the user's search history.

They are used to predict the rating or preference that a user would give to an item

Examples:

Spotify, Netflix → they rely on the effectiveness of their recommendation engines for their business and success.

YouTube → uses them to decide which video out **of ~ca billion films** to play during → autoplay,

Facebook → uses it to recommend pages to like and follow.

Amazon → uses them to suggest products to customers.

The Google Play Store offers **millions of apps** → QUESTION : → How can users find new, attractive content?
Answer → Yes, you can use search to access content. However, the recommendation engine may display items that users might not think about on their own.

… other …

Give examples yourself

What are the goals of recommendation systems?

https://developers.google.com/machine-learning/recommendation

Description of the model construction:

https://developers.google.com/machine-learning/recommendation

The goals of recommendation systems are:

… predicting user preferences and offering them:

- products → … Content or → services → that may interest them

… supporting you in making decisions in a crowded information world, where the number of available options can be overwhelming

For the user, the goals are therefore:

…. increasing user satisfaction

…. improving the shopping experience

…. and promoting longer and more engaged interaction with a given website or platform

For the company, the goals are:

…. they help increase sales

…. maintaining customer loyalty

…. optimization of the offer

What are the components of a recommendation system?

https://developers.google.com/machine-learning/recommendation

Description of the model construction:

https://developers.google.com/machine-learning/recommendation

Components of the recommendation system:

1. Data collection

Recommendation systems start by collecting data about users, their preferences, purchase history, ratings, and even demographic data.

2. Data preprocessing:

The data is then processed and cleansed to remove errors, missing data and other irregularities.

3. Candidate generation model:

the component is responsible for creating an initial list of proposals (candidates) based on the analysis of user preferences. This may include content-based methods (content analysis) or collaborative filtering (analysis of similarity between users or items).

4. Candidate evaluation:

The system must evaluate how well each candidate fits the user's preferences. This rating may be based on a user's previous interactions, ratings from other users, or other factors.

5. Model update:

As the user → uses the system, preferences change, and the model must be updated regularly to reflect these changes.

6. Recommendation generation:

Based on the candidates' ratings, the system generates final recommendations for the user.

7. Positioning in the ranking:

If the system generates more than one recommendation, it is necessary to determine which one will be most attractive to the user. This ranking positioning takes into account additional factors such as product popularity, availability and freshness.

8. User feedback:

The feedback collected from users is valuable for further improvement of the system. The recommendation system must be flexible and adapt to changing user preferences and expectations.

https://developers.google.com/machine-learning/recommendation

Terminology

Description of the model construction:

https://developers.google.com/machine-learning/recommendation

Terminology

Items (also called documents)

Items → recommended by the system. In the case of the Google Play Store, items → are applications to be installed. For YouTube, items → are films.

Query (context):

→ Information that the system uses to create recommendations. Queries can be a combination of these elements.

Placement (embedding)

A mapping from a specific set (in this case, a set of queries or recommended items) → to a vector space called an embedding space. Many recommendation systems require learning to represent queries and items appropriately.

https://grouplens.org/datasets/movielens/

grouplens

Social Computing Research at the University of Minnesota

movielens

Non-commercial, personalized movie recommendatio

Data Set Information:

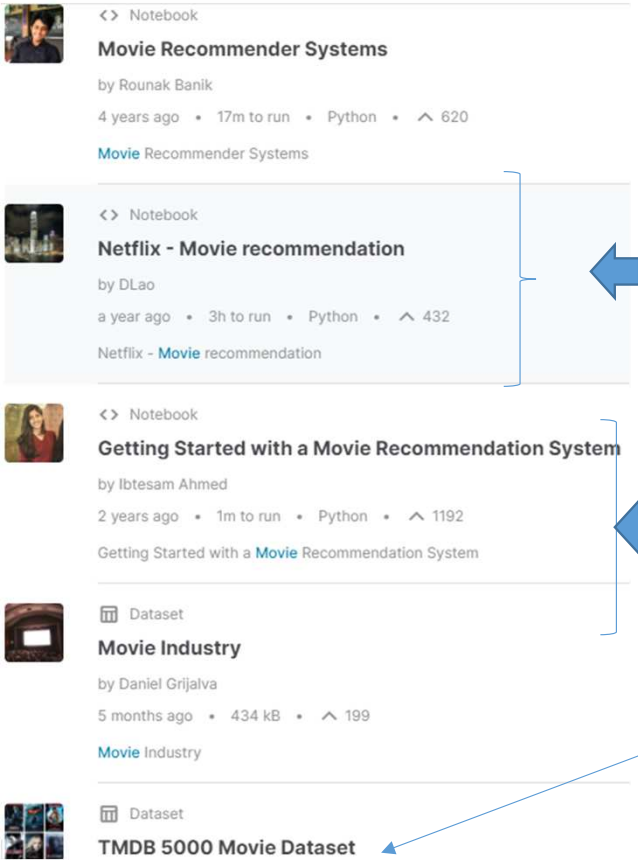
GroupLens Research has collected and shared ratings datasets from the website: **MovieLens** (https://movielens.org).
The datasets were collected over different periods of time, depending on the size of the dataset.

ATTENTION !!!:

Before using these datasets, review files and README for usage licenses and other details.

https://www.kaggle.com/search?q=movie

Kaggle → Data sets



Movie Recommendation through Correlations / CF

The dataset, directly from Netflix, consists of 4 text data files and contains over 20M rows, for over 4K movies and 400K clients. Total over 17 thousand films and over 500 thousand customers!

Movie **recommendation system** using the **TMDB 5000** movie dataset.

Demographic Filtering - recommendations based on movie popularity and/or genre. The system recommends the same videos to users with similar demographic characteristics. Videos that are more popular and appreciated by a group of people will be more likely to be liked than those selected by the average audience.

Content Based Filtering- Suggest similar items based on a specific item. uses movie metadata such as genre, director, description, actors, etc. Recommendation based on → that if a person likes a specific item, he or she will also like an item that is similar to it.

Collaborative Filtering - matches people with similar interests and provides recommendations based on that match. Collaborative filters do not require content-based element metadata.

https://www.kaggle.com/ibtesama/getting-started-with-a-movie-recommendation-system

Getting Started with a Movie Recommendation System

Movie recommendation system using the **TMDB 5000** movie dataset.

Demographic Filtering:

Required metrics: → … for scoring or rating the film:

A. Calculates the metric scores for each movies

B. Sorting results by metrics, e.g.:

average movie ratings as scores → but using the average alone is not sufficient → because a movie with an average rating of 8.9 and only 3 votes is not better than a movie with an average rating of 7.8 but 40 votes.

For this reason, a **weighted average rating** (wr) is used, which is given as:

$$\text{Weighted Rating (WR)} = \left(\frac{v}{v+m} . R\right) + \left(\frac{m}{v+m} . C\right)$$

w here :

v → number of votes for the film

m → is the minimum number of votes required to be included in the table

R → is the average rating of the film;

C → is the average number of votes in the entire report

C. Recommendation (limit N) of the best rated movies

https://www.kaggle.com/ibtesama/getting-started-with-a-movie-recommendation-system

Getting Started with a Movie Recommendation System

Movie recommendation system using the **TMDB 5000** movie dataset.

Content Based Filtering:

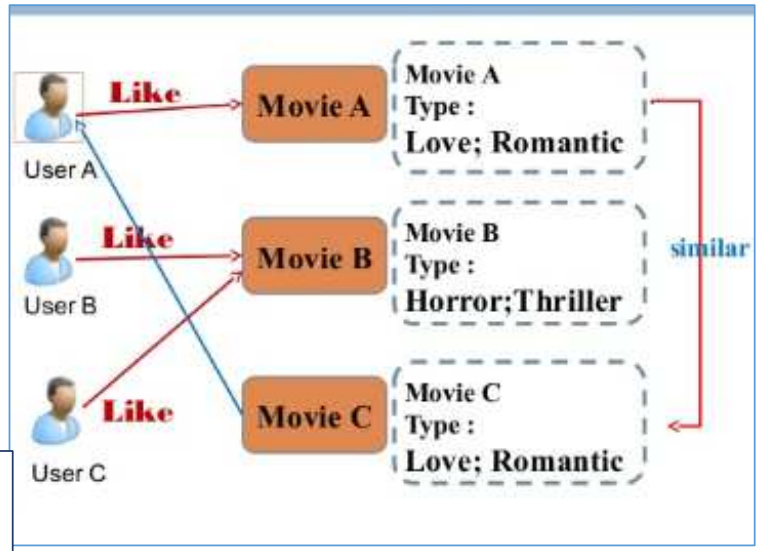
→ suggest similar items based on a specific item. uses movie metadata such as genre, director, description, actors, etc.
Recommendation based → that if a person likes a particular item, he or she will also like an item that is similar to it.

A.

It involves finding similarity in movie recommendations based on the information provided about the movie's content (review, cast, crew, keyword, tagline, etc.)

We will compute pairwise similarity scores for all movies based on their plot descriptions and .

→… we will recommend movies based on this similarity score. A description of the plot is provided in the overview function of our dataset.



... for this purpose, the text must be converted into a vector of words for each description and then analyzed, e.g. **Term Frequency-Inverse Document** - they analyze the number of word occurrences and their semanticity.

this is the relative frequency of a word in the document and is given as (occurrence term/total number of occurrences). Inverse document frequency is the relative number of documents containing a term given as log(number of documents/documents with term)

Over 20,000 different words were used to describe the 4,800 videos in the **TMDB 5000** dataset.

Having built the matrix → a similarity score can be calculated. There are several algorithms for this:

similarity rating:

- Euclidean,
- Pearson
- cosine.

B.

Movies that are most likely to be similar are recommended.

https://developers.google.com/machine-learning/recommendation/overview/candidate-generation

Question :

Which recommendation model should I choose?

the answer

VIDE next →

https://developers.google.com/machine-learning/recommendation/overview/candidate-generation

Which recommendation model should I choose?

The answer is… → There is no clear answer because it depends...

Question: → .. what does this depend on?

From **queries** sent to the system → that is… → from their **context**

The information the system uses to create recommendations is based on queries, which may be a combination of the following elements:

We have user information:
- user ID
- elements that users have previously interacted with
- …other..

… and additional context:
- time of day
- user device
- …other..

Placement (embedding)

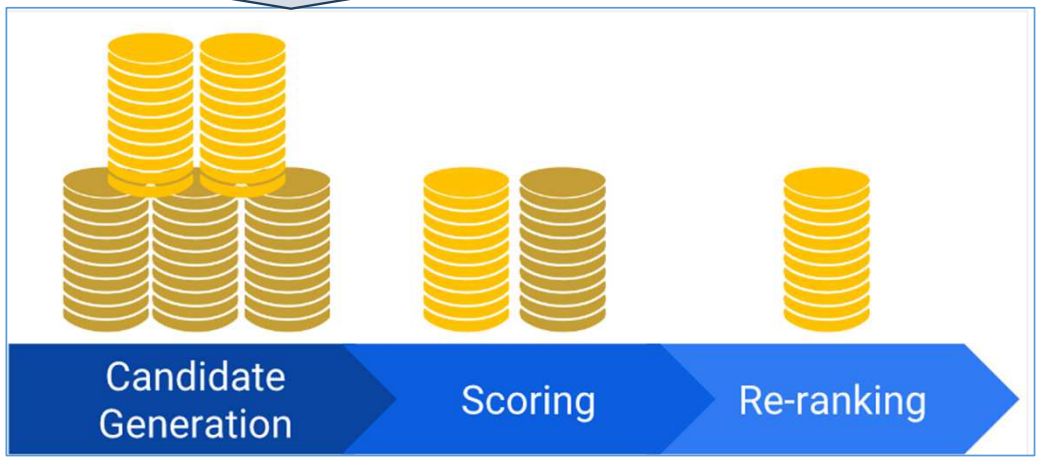
VIDE dalej → embedded space

B

A mapping from a specific set (in this case, a set of queries or recommended items) to a vector space called an embedding space. Many recommendation systems require learning to represent queries and items appropriately.

https://developers.google.com/machine-learning/recommendation/overview/candidate-generation

One of the typical architectures of recommendation systems consists of the following elements:



A. Candidate Generation Overview

In this first stage, the system starts from a potentially huge corpus and generates a much smaller subset of candidates.

For example, the candidate generator in YouTube reduces billions of videos down to hundreds or thousands.

The model needs to evaluate queries quickly given the enormous size of the corpus. A given model may provide multiple candidate generators, each nominating a different subset of candidates.

B. Scoring

Next, another model scores and ranks the candidates in order to select the set of items (on the order of 10) to display to the user. Since this model evaluates a relatively small subset of items, the system can use a more precise model relying on additional queries.

C. Re-ranking

Finally, the system must take into account additional constraints for the final ranking. For example, the system removes items that the user explicitly disliked or boosts the score of fresher content. Re-ranking can also help ensure diversity, freshness, and fairness.

https://developers.google.com/machine-learning/recommendation/overview/candidate-generation

A. Candidate Generation:

Candidate generation is the first stage of recommendation. Given a query, the system generates a set of relevant candidates.

The following table shows two common candidate generation approaches:

Type	Definition	Example
content-based filtering	Uses *similarity between items* to recommend items similar to what the user likes.	If user A watches two cute cat videos, then the system can recommend cute animal videos to that user.
collaborative filtering	Uses *similarities between queries and items simultaneously* to provide recommendations.	If user A is similar to user B, and user B likes video 1, then the system can recommend video 1 to user A (even if user A hasn't seen any videos similar to video 1).

https://developers.google.com/machine-learning/recommendation/overview/candidate-generation

B **Embedding Space**

Both content-based and collaborative filtering map each item and each query (or context) to an embedding vector in a common embedding space $E = \mathbb{R}^d$

Typically, the embedding space is low-dimensional (that is ***d*** is much smaller than the size of the corpus), and captures some latent structure of the item or query set. Similar items, such as YouTube videos that are usually watched by the same user, end up close together in the embedding space. The notion of "closeness" is defined by a similarity measure.

Similarity Measures

A similarity measure is a function: $s : E \times E \to \mathbb{R}$ that takes a pair of embeddings and returns a scalar measuring their similarity.

The embeddings can be used for candidate generation as follows: given a query embedding: $q \in E$

the system looks for item embeddings: $x \in E$ that are close to ***q*** that is, embeddings with high similarity $s(q, x)$

To determine the degree of similarity, most recommendation systems rely on one or more of the following:

- cosine
- dot product
- Euclidean distance

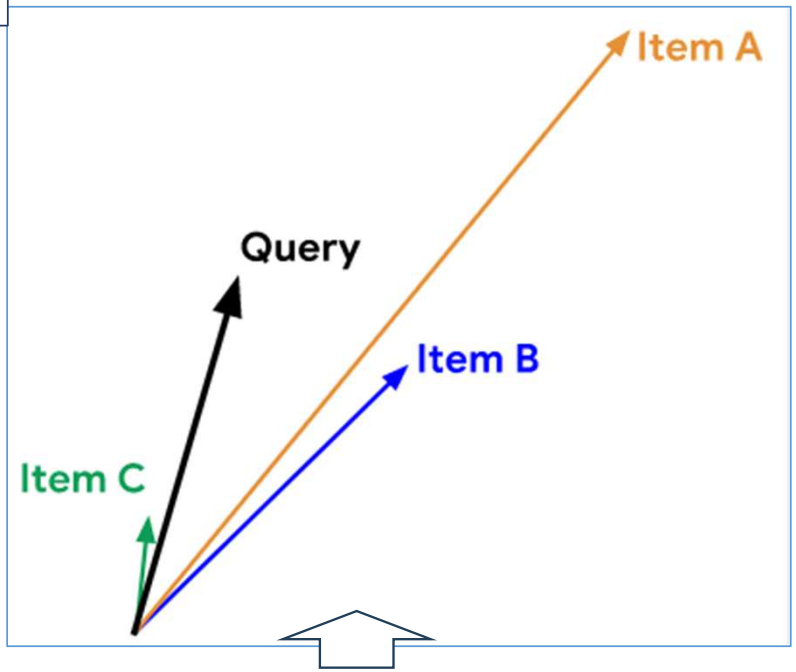
https://developers.google.com/machine-learning/recommendation/overview/candidate-generation

Cosine: This is simply the cosine of the angle between the two vectors: $s(q, x) = \cos(q, x)$

Dot Product: The dot product of two vectors is: $s(q, x) = \langle q, x \rangle = \sum_{i=1}^{d} q_i x_i$

It is also given by: $s(q, x) = \|x\| \|q\| \cos(q, x)$

(the cosine of the angle multiplied by the product of norms). Thus, if the embeddings are normalized, then dot-product and cosine coincide.



Euclidean distance:

This is the usual distance in Euclidean space:

$$s(q, x) = \|q - x\| = \left[\sum_{i=1}^{d} (q_i - x_i)^2\right]^{\frac{1}{2}}$$

smaller distance means higher similarity. Note that when the embeddings are normalized, the squared Euclidean distance coincides with dot-product (and cosine) up to a constant, since in that case: $\frac{1}{2}\|q - x\|^2 = 1 - \langle q, x \rangle$.

Comparing Similarity Measures:

The black vector illustrates the query embedding. The other three embedding vectors (Item A, Item B, Item C) represent candidate items. Depending on the similarity measure used, the ranking of the items can be different.

https://www.kaggle.com/ibtesama/getting-started-with-a-movie-recommendation-system

Getting Started with a Movie Recommendation System

Movie recommendation system using the **TMDB 5000** movie dataset.

Content Based Filtering: ⇨ similarity rating: ⇨ **cosinusowa.**

cosine similarity score since it is independent of magnitude and is relatively easy and fast to calculate. Mathematically, it is defined as follows:

Mathematical description:

$$\text{similarity} = \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2}\sqrt{\sum_{i=1}^{n} B_i^2}},$$

Procedure:

Get the index of the film by entering its title.

Get a list of cosine similarity scores for this specific video with all videos.

Convert it into a list of tuples, where the first element is its position and the second is its similarity score.

Sort the above-mentioned list of tuples based on similarity scores; that is, the second element.

Get the top 10 items from this list.

Ignore the first item because it refers to itself (the movie most similar to a specific movie is the movie itself).

Return titles corresponding to the indexes of the top elements.

https://www.kaggle.com/ibtesama/getting-started-with-a-movie-recommendation-system

Getting Started with a Movie Recommendation System

Movie recommendation system using the **TMDB 5000** movie dataset.

Collaborative Filtering
(Filtrowanie grupowe)

It uses similarities between queries and items simultaneously to make recommendations.

If user A is similar to user B and user B likes movie 1, the system can recommend movie 1 to user A (even if user A has not seen any movies similar to movie 1).

https://www.kaggle.com/ibtesama/getting-started-with-a-movie-recommendation-system

Example of a movie recommendation:

Consider a movie recommendation system where the training data consists of a feedback matrix in which:

- Each row represents a user.
- Each column represents an item (movie).

The feedback about movies falls into one of two categories:

- **Explicit** — users specify how much they liked a particular movie by providing a numerical rating.
- **Implicit** — if a user watches a movie, the system infers that the user is interested.

To simplify, we will assume that the feedback matrix is binary; that is, a value of 1 indicates interest in the movie.

When a user visits the homepage, the system should recommend movies based on both:

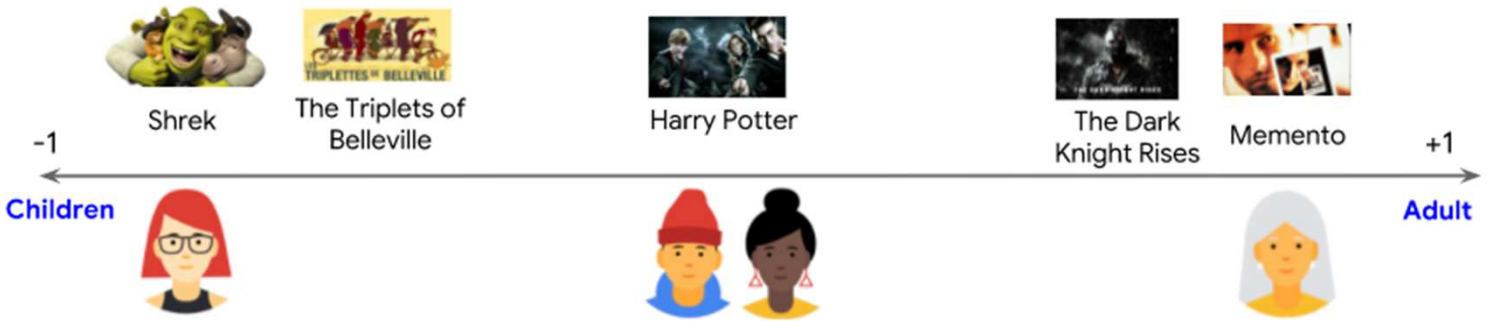
- similarity to movies the user has liked in the past
- movies that similar users liked

For the sake of illustration, let's hand-engineer some features for the movies described in the following table:

Movie	Rating	Description
The Dark Knight Rises	PG-13	Batman endeavors to save Gotham City from nuclear annihilation in this sequel to The Dark Knight, set in the DC Comics universe.
Harry Potter and the Sorcerer's Stone	PG	A orphaned boy discovers he is a wizard and enrolls in Hogwarts School of Witchcraft and Wizardry, where he wages his first battle against the evil Lord Voldemort.
Shrek	PG	A lovable ogre and his donkey sidekick set off on a mission to rescue Princess Fiona, who is emprisoned in her castle by a dragon.
The Triplets of Belleville	PG-13	When professional cycler Champion is kidnapped during the Tour de France, his grandmother and overweight dog journey overseas to rescue him, with the help of a trio of elderly jazz singers.
Memento	R	An amnesiac desperately seeks to solve his wife's murder by tattooing clues onto his body.

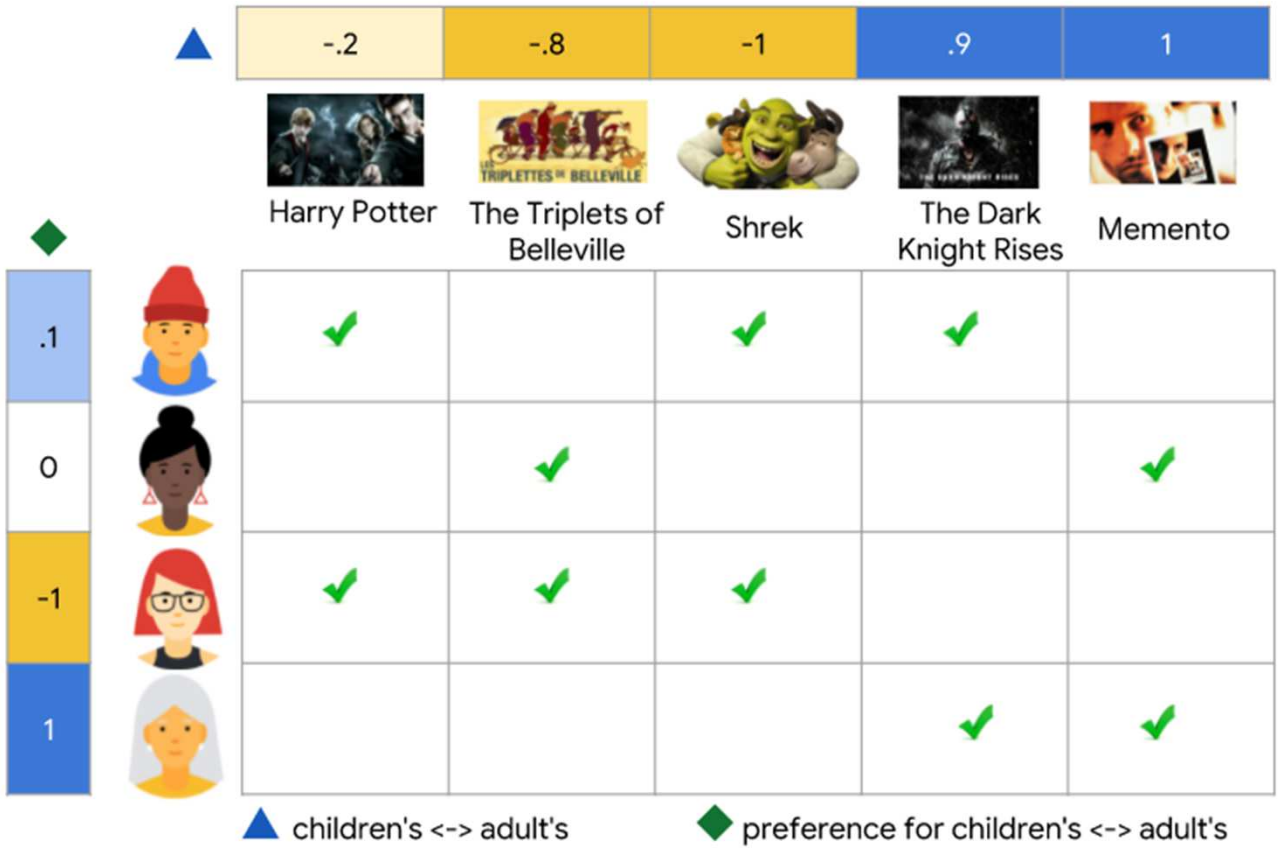
1D Embedding

Suppose we assign to each movie a scalar in **[-1, 1]** that describes whether the movie is for children (negative values) or adults (positive values). Suppose we also assign a scalar to each user in **[-1,1]** that describes the user's interest in children's movies (closer to -1) or adult movies (closer to +1). The product of the movie embedding and the user embedding should be higher (closer to 1) for movies that we expect the user to like.



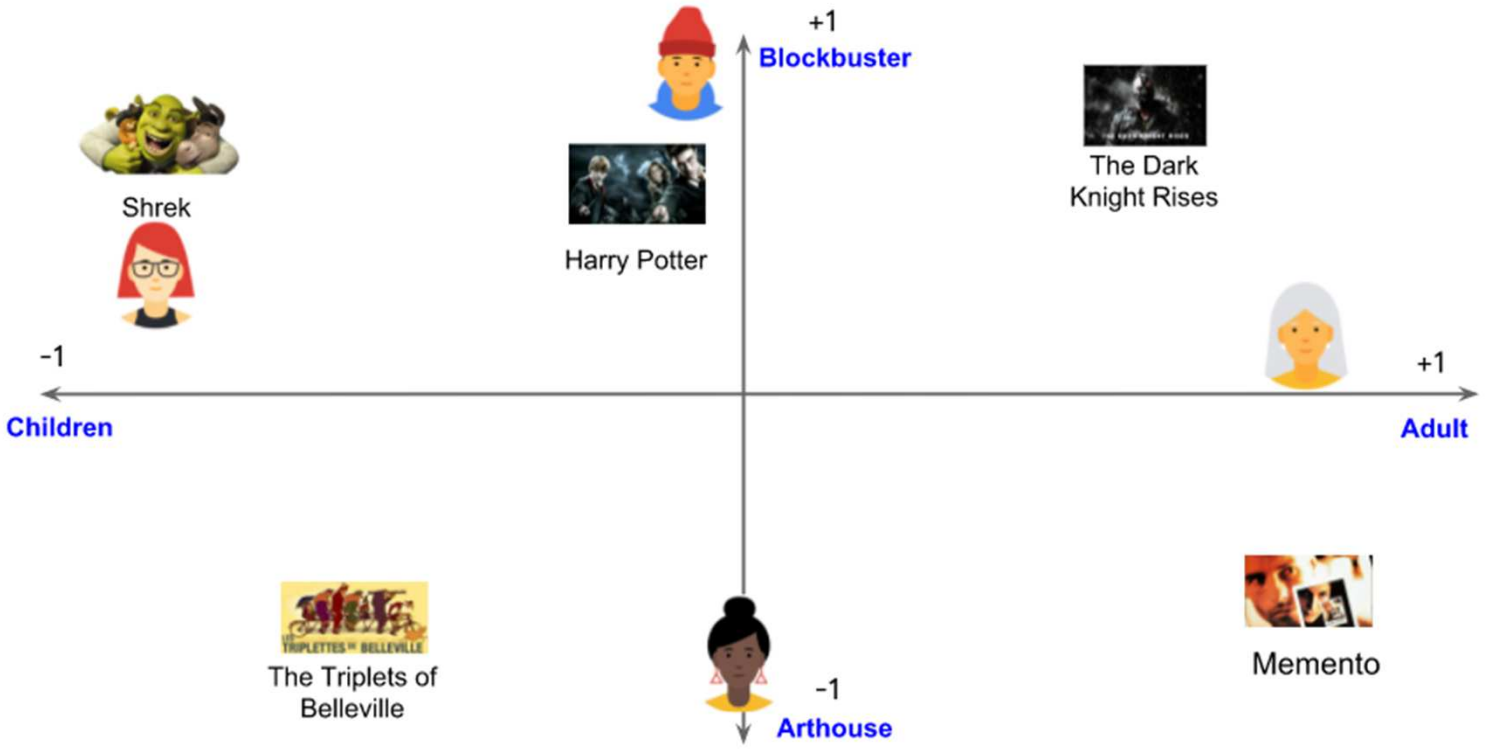
1D Embedding

In the diagram below, each checkmark identifies a movie that a particular user watched. The third and fourth users have preferences that are well explained by this feature—the third user prefers movies for children and the fourth user prefers movies for adults. However, the first and second users' preferences are not well explained by this single feature.



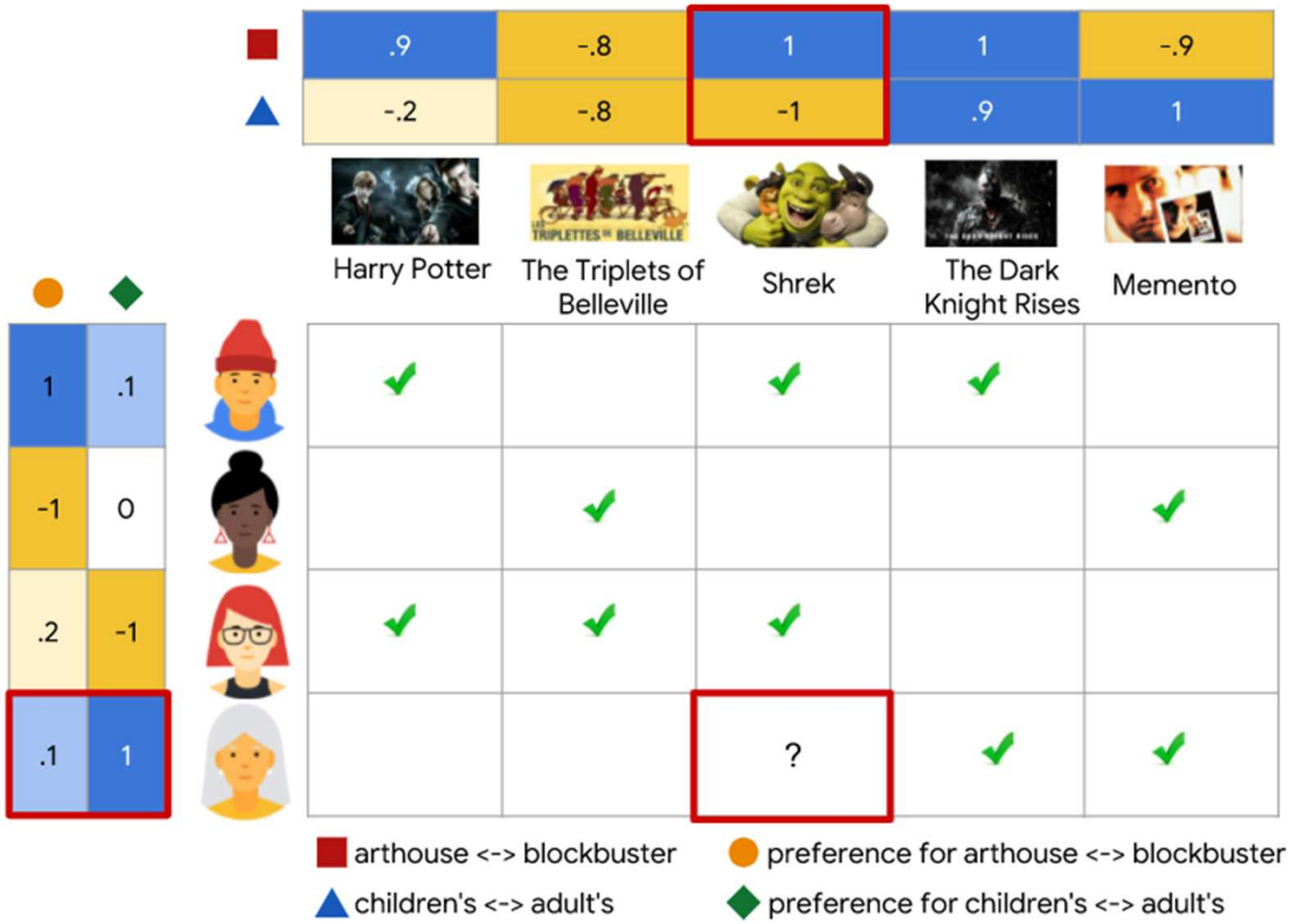
Dla ilustracji opracujmy ręcznie niektóre funkcje filmów opisanych w poniższej tabeli:

2D Embedding	One feature was not enough to explain the preferences of all users. To overcome this problem, let's add a second feature: the degree to which each movie is a blockbuster or an arthouse movie. With a second feature, we can now represent each movie with the following two-dimensional embedding:



2D Embedding

We again place our users in the same embedding space to best explain the feedback matrix: for each (user, item) pair, we would like the dot product of the user embedding and the item embedding to be close to 1 when the user watched the movie, and to 0 otherwise.



Note:

We represented both items and users in the same embedding space. This may seem surprising. After all, users and items are two different entities. However, you can think of the embedding space as an abstract representation common to both items and users, in which we can measure similarity or relevance using a similarity metric.

The collaborative nature of this approach is apparent when the model learns the embeddings. Suppose the embedding vectors for the movies are fixed. Then, the model can learn an embedding vector for the users to best explain their preferences. favor, embeddings of users with similar preferences will be close together. similarly, if the embeddings for the users are fixed, then we can learn movie embeddings to best explain the feedback matrix. As a result, embeddings of movies liked by similar users will be close in the embedding space.

Matrix Factorization:

Matrix factorization is a simple embedding model. Given the feedback matrix $A \in R^{m \times n}$

where:
m → is the number of users (or queries) and
n → is the number of items, the model learns:

A user embedding matrix: $U \in \mathbb{R}^{m \times d}$ where row i is the embedding for user i.

An item embedding matrix $V \in \mathbb{R}^{n \times d}$ where row j is the embedding for item j.



The embeddings are learned such that the product: UV^T is a good approximation of the feedback matrix A. Observe that the : (i, j)

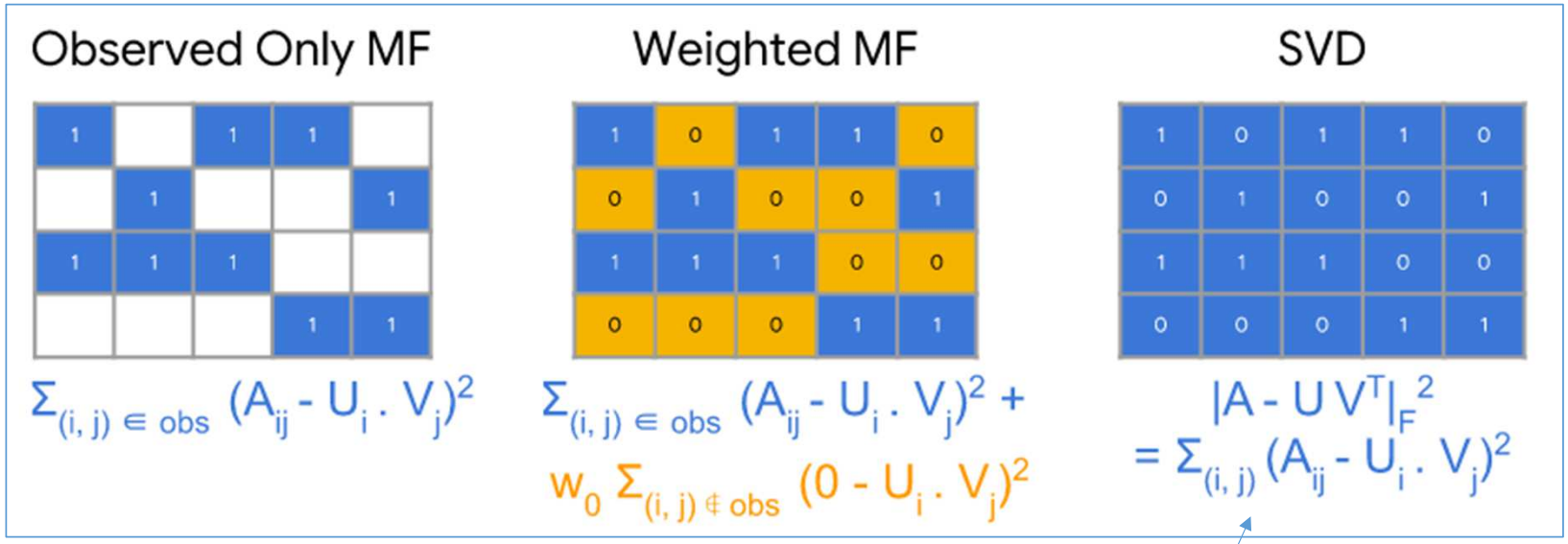
entry of: $U.V^T$ is simply the dot product: $\langle U_i, V_j \rangle$ of the embeddings of user ***I*** and item ***j*** which you want to be close to: $A_{i,j}$.

Choosing the Objective Function:

One intuitive objective function is the squared distance. To do this, minimize the sum of squared errors over all pairs of observed entries:

$$\min_{U \in \mathbb{R}^{m \times d},\, V \in \mathbb{R}^{n \times d}} \sum_{(i,j) \in \text{obs}} (A_{ij} - \langle U_i, V_j \rangle)^2.$$

In this objective function, you only sum over observed pairs (i, j), that is, over non-zero values in the feedback matrix. However, only summing over values of one is not a good idea—a matrix of all ones will have a minimal loss and produce a model that can't make effective recommendations and that generalizes poorly.



decomposition into singular values (SVD)

Choosing the Objective Function:

You can solve this quadratic problem through Singular Value Decomposition (SVD) of the matrix. However, SVD is not a great solution either, because in real applications, the matrix ***A*** may be very sparse. For example, think of all the videos on YouTube compared to all the videos a particular user has viewed.

The solution: UV^T (which corresponds to the model's approximation of the input matrix) will likely be close to zero, leading to poor generalization performance.

In contrast, **Weighted Matrix Factorization** decomposes the objective into the following two sums:

- A sum over observed entries.
- A sum over unobserved entries (treated as zeroes).

$$\min_{U\in\mathbb{R}^{m\times d},\, V\in\mathbb{R}^{n\times d}} \sum_{(i,j)\in\text{obs}} (A_{ij} - \langle U_i, V_j\rangle)^2 + w_0 \sum_{(i,j)\notin\text{obs}} (\langle U_i, V_j\rangle)^2.$$

Here **w0** is a hyperparameter that weights the two terms so that the objective is not dominated by one or the other. Tuning this hyperparameter is very important.

SVD

1	0	1	1	0
0	1	0	0	1
1	1	1	0	0
0	0	0	1	1

$$|A - U V^T|_F^2 = \Sigma_{(i,j)} (A_{ij} - U_i \cdot V_j)^2$$

Minimizing the Objective Function:

Common algorithms to minimize the objective function include:

Stochastic gradient descent (SGD) → is a generic method to minimize loss functions.

Weighted Alternating Least Squares (WALS) → is specialized to this particular objective.

The objective is quadratic in each of the two matrices U and V. (Note, however, that the problem is not jointly convex.) WALS works by initializing the embeddings randomly, then alternating between:

Fixing ***U*** and solving for ***V***

Fixing ***V*** and solving for ***U***

SGD vs. WALS

Common algorithms to minimize the objective function include:

SGD



Very flexible—can use other loss functions.



Can be parallelized.



Slower—does not converge as quickly.



Harder to handle the unobserved entries (need to use negative sampling or gravity).

WALS



Can be parallelized.



Converges faster than SGD.



Easier to handle unobserved entries.



Reliant on Loss Squares only.

https://grouplens.org/datasets/movielens/

grouplens | Social Computing Research at the University of Minnesota

movielens
Non-commercial, personalized movie recommendations.

Data Set Information:

GroupLens → research collected and shared assessment datasets from the website **MovieLens** (https://movielens.org).
The datasets were collected over different periods of time, depending on the size of the dataset.
Before using these datasets, review their files and README for license of use and other details.

*Small **MovieLens** datasets:* 100,000 ratings and 3,600 tag applications applied to 9,000 movies by 600 users. Last updated 9/2018.

Description:

- Users were randomly selected for inclusion.
- All selected users rated at least 20 videos.
- No demographic data available.
- Each user is represented by an ID and no other information is provided.

Mały : 100 000 ocen i 3600 aplikacji tagów
aktualizacja 9/2018.

- README.html
- ml-latest-small.zip (rozmiar: 1 MB)

Data contained in the files:

links.csv, **movies.csv**, **ratings.csv**, **tags.csv**.

ratings.csv — **UTF-8** encoding

userId	User ID
movieId	Movie ID
rating	Video rating from 0 to 5 in 0.5 increments
timestamp	Time in seconds (epochs) from 1970/01/01

movies.csv

movieId	movie ID
title	Title of the movie
genres	The species are a list separated by a vertical bar and are as follows:

* Action* Adventure* Animation* Children's* Comedy* Crime* Documentary* Drama* Fantasy* Film-Noir* Horror* Musical* Mystery* Romance* Sci-Fi* Thriller* War* Western* (no genres listed)

```
movieId,title,genres
1,Toy Story (1995),Adventure|Animation|Children|Comedy|Fantasy
2,Jumanji (1995),Adventure|Children|Fantasy
3,Grumpier Old Men (1995),Comedy|Romance
4,Waiting to Exhale (1995),Comedy|Drama|Romance
5,Father of the Bride Part II (1995),Comedy
```

mlr_1rec.py

We start by reading the set:

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("ml").getOrCreate()
ratings = spark.read.csv("ml-latest-small/ratings.csv", sep=",", header=True, inferSchema=True)
ratings.show(10)
ratings.printSchema()
```

We allow data types to be recognized

```
+------+-------+------+----------+
|userId|movieId|rating| timestamp|
+------+-------+------+----------+
|     1|      1|   4.0|964982703|
|     1|      3|   4.0|964981247|
|     1|      6|   4.0|964982224|
|     1|     47|   5.0|964983815|
|     1|     50|   5.0|964982931|
|     1|     70|   3.0|964982400|
```

Time in epochs in seconds from January 1, 1970

```
root
 |-- userId: integer (nullable = true)
 |-- movieId: integer (nullable = true)
 |-- rating: double (nullable = true)
 |-- timestamp: integer (nullable = true)
```

VIDE slide next

mlr_2rec.py

cd.VIDE slide next

We start by reading the set:

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("ml").getOrCreate()
ratings = spark.read.csv("ml-latest-small/ratings.csv", sep=",", header=True, inferSchema=True)
train, test = ratings.randomSplit([0.8, 0.2],42)
```

We divide our set into two subsets: First → for training, the second → for testing

Parameter 42 → specifies the randomization seed

```
+------+-------+------+---------+
|userId|movieId|rating|timestamp|
+------+-------+------+---------+
|     1|      1|   4.0|964982703|
|     1|      3|   4.0|964981247|
|     1|      6|   4.0|964982224|
|     1|     47|   5.0|964983815|
|     1|     50|   5.0|964982931|
|     1|     70|   3.0|964982400|
```

Time in epochs in seconds from January 1, 1970

```
root
 |-- userId: integer (nullable = true)
 |-- movieId: integer (nullable = true)
 |-- rating: double (nullable = true)
 |-- timestamp: integer (nullable = true)
```

VIDE slide next

mlr_2rec.py

We start by reading the set:

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("ml").getOrCreate()
ratings = spark.read.csv("ml-latest-small/ratings.csv", sep=",", header=True, inferSchema=True)
train, test = ratings.randomSplit([0.8, 0.2],42)
from pyspark.ml.recommendation import ALS
als = ALS(maxIter=5, regParam=0.01, userCol="userId", itemCol="movieId", ratingCol="rating",
        seed=42, coldStartStrategy="drop")
model = als.fit(train)
pred = model.transform(test)
pred.show(10)
```

(ALS) - **a**lternating **l**east **s**quares → algorithm to learn these latent factors.

We import a specific library for the development of a recommendation model

It initializes the estimator - i.e. we prepare a ready-made, untrained model

MaxIter – the model will go through 5 iterations → to train

Regularization parameter

Next → the user ID, video and rating columns

Seed → set a random seed for model training

coldStartStrategy → set no recommendation strategies for the new user

The estimator requires the use of a method **fit**

In the output of the **fit()** method we have a transformer, which provides the transformer method with the trained model

The **transform** method allows for predictions

Prediction→ tries to bring prediction and rating closer

```
+------+-------+------+----------+----------+
|userId|movieId|rating| timestamp|prediction|
+------+-------+------+----------+----------+
|   182|    471|   4.5|1054779644|  3.484336|
|   462|    471|   2.5|1123890831| 3.9725325|
|   610|    471|   4.0|1479544381| 3.4282506|
|   176|    471|   5.0| 840109075| 4.5944266|
|   448|    471|   4.0|1178980875| 2.9242966|
```

Spark allows you to set the **coldStartStrategy** parameter to "drop" to drop any rows in the **DataFrame** that contain NaN values in your forecasts. The evaluation metric will then be calculated on the non-data NaN and will be valid.

VIDE slide next

mlr_2rec.py

Evaluation

We start the evaluation by importing the library: RegressionEvaluator

mlr_3rec.py

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("ml").getOrCreate()
ratings = spark.read.csv("ml-latest-small/ratings.csv", sep=",", header=True, inferSchema=True)
train, test = ratings.randomSplit([0.8, 0.2],42)
from pyspark.ml.recommendation import ALS
als = ALS(maxIter=5, regParam=0.01, userCol="userId", itemCol="movieId", ratingCol="rating",
      seed=42, coldStartStrategy="drop")
model = als.fit(train)
pred = model.transform(test)
from pyspark.ml.evaluation import RegressionEvaluator
evaluator = RegressionEvaluator(metricName="rmse", labelCol="rating", predictionCol="prediction")
evaluator.evaluate(pred)
```

```
1.0823243058259922
```

VIDE slide next

mlr_2rec.py

Films worth showing to users:

VIDE slide next

We use the method: recommendForAllUsers

mlr_3rec.py

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("ml").getOrCreate()
ratings = spark.read.csv("ml-latest-small/ratings.csv", sep=",", header=True, inferSchema=True)
train, test = ratings.randomSplit([0.8, 0.2],42)
from pyspark.ml.recommendation import ALS
als = ALS(maxIter=5, regParam=0.01, userCol="userId", itemCol="movieId", ratingCol="rating",
      seed=42, coldStartStrategy="drop")
model = als.fit(train)
pred = model.transform(test)
from pyspark.ml.evaluation import RegressionEvaluator
evaluator = RegressionEvaluator(metricName="rmse", labelCol="rating", predictionCol="prediction")
evaluator.evaluate(pred)
model.recommendForAllUsers(3).show(10, truncate=False)
```

https://spark.apache.org/docs/latest/ml-collaborative-filtering.html

https://stackoverflow.com/questions/31544691/how-to-recommend-top-10-products-in-spark-als-for-all-the-users

```
+------+--------------------------------------------------------------------+
|userId|recommendations                                                     |
+------+--------------------------------------------------------------------+
|471   |[{69406, 8.0032625}, {96004, 7.845047}, {32598, 7.700263}]          |
|463   |[{171763, 6.701351}, {5319, 6.510456}, {92535, 6.441631}]           |
|496   |[{3265, 7.251126}, {85780, 6.840547}, {97225, 6.8291626}]           |
|148   |[{56941, 7.0331554}, {118900, 6.72642}, {3030, 6.7040443}]          |
|540   |[{7046, 6.869965}, {3435, 6.7172174}, {171763, 6.415837}]           |
|392   |[{1334, 10.1420765}, {1194, 9.919676}, {140711, 8.537537}]          |
|243   |[{103341, 11.4821825}, {1260, 11.4727125}, {2613, 10.853818}]       |
|31    |[{102903, 12.082401}, {93270, 11.635988}, {179819, 11.196424}]      |
|516   |[{2290, 8.513212}, {166534, 7.9887304}, {1904, 7.8702803}]          |
|580   |[{82, 6.6438103}, {6818, 6.405214}, {71899, 5.6740885}]             |
+------+--------------------------------------------------------------------+
```

VIDE slide next

mlr_2rec.py

Users who are most likely to like these movies:

VIDE slide next

Users who are most likely to like these movies:

https://spark.apache.org/docs/2.2.0/ml-collaborative-filtering.html

mlr_3rec.py

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("ml").getOrCreate()
ratings = spark.read.csv("ml-latest-small/ratings.csv", sep=",", header=True, inferSchema=True)
train, test = ratings.randomSplit([0.8, 0.2],42)
from pyspark.ml.recommendation import ALS
als = ALS(maxIter=5, regParam=0.01, userCol="userId", itemCol="movieId", ratingCol="rating",
      seed=42, coldStartStrategy="drop")
model = als.fit(train)
pred = model.transform(test)
from pyspark.ml.evaluation import RegressionEvaluator
evaluator = RegressionEvaluator(metricName="rmse", labelCol="rating", predictionCol="prediction")
evaluator.evaluate(pred)
model.recommendForAllItems(3).show(10, truncate=False)
```

```
+------+--------------------------------------------------------------+
|userId|recommendations                                               |
+------+--------------------------------------------------------------+
|471   |[{69406, 8.0032625}, {96004, 7.845047}, {32598, 7.700263}]    |
|463   |[{171763, 6.701351}, {5319, 6.510456}, {92535, 6.441631}]     |
|496   |[{3265, 7.251126}, {85780, 6.840547}, {97225, 6.8291626}]     |
|148   |[{56941, 7.0331554}, {118900, 6.72642}, {3030, 6.7040443}]    |
|540   |[{7046, 6.869965}, {3435, 6.7172174}, {171763, 6.415837}]     |
|392   |[{1334, 10.1420765}, {1194, 9.919676}, {140711, 8.537537}]    |
|243   |[{103341, 11.4821825}, {1260, 11.4727125}, {2613, 10.853818}] |
|31    |[{102903, 12.082401}, {93270, 11.635988}, {179819, 11.196424}]|
|516   |[{2290, 8.513212}, {166534, 7.9887304}, {1904, 7.8702803}]    |
|580   |[{82, 6.6438103}, {6818, 6.405214}, {71899, 5.6740885}]       |
+------+--------------------------------------------------------------+
```

VIDE slide next

mlr_2rec.py

Top 5 moviess for specific users

VIDE slide next

Users who are most likely to like these movies:

mlr_3rec.py

own homework → search on the Apache Spark project website

VIDE slide next